



Saint Columban College
College of Computing Studies
Pagadian City



PW Crack 2

PW Crack 2 



Easy

General Skills

Beginner picoMini 2022

password_cracking

AUTHOR: LT 'SYREAL' JONES

Description

Can you crack the password to get the flag?

Download the password checker [here](#) and you'll need the encrypted [flag](#) in the same directory too.

Hints 

1

2

Does that encoding look familiar?

69,205 users solved



93%

Liked



 picoCTF{tr45h_51ng1ng_502ec42e}

Submit
Flag

Author: The Analyst: Hyposelenia

Challenge: PW Crack 2

Category: General Skills

Date: 01/27/26



I. Objective

The objective of this challenge is to analyze a provided Python script to determine the correct password required to decrypt an encrypted file and retrieve the hidden flag.

II. Background

Password-related challenges in Capture The Flag (CTF) competitions are designed to introduce participants to **basic authentication logic, encoding schemes, and secure coding concepts**. Rather than relying on brute-force attacks, beginner challenges typically emphasize **source code inspection and logical analysis**.

In this challenge, a Python script checks a user-provided password before decrypting an encrypted text file. Unlike PW Crack 1, the correct password is **not directly visible in plain text**. Instead, it is stored in **hexadecimal format**, requiring decoding before it can be used. This highlights the importance of understanding common encoding methods such as hexadecimal and ASCII.

III. Tool Used

- **Oracle VirtualBox (Kali Linux)** – Used as the Linux environment
- **Linux Terminal** – Used to execute decoding commands
- **Python 3** – Used to execute the provided Python script
- **nano** – Used to view and inspect the Python source code
- **Online Hex to ASCII Converter** – Used to decode the hexadecimal password
(<https://www.rapidtables.com/convert/number/ascii-hex-bin-dec-converter.html>)

IV. Methodology

1. Opened Oracle VirtualBox and launched Kali Linux.



- Downloaded the provided challenge files, which included a Python script and an encrypted text file, by copying the address link.

```
kali@kali: /media/sf_Downloaded_Cybersecfiles
File Actions Edit View Help
--(kali@kali): /media/sf_Downloaded_Cybersecfiles
$ wget https://artifacts.picoctf.net/c/15/level2.py
2020-01-27 05:53:15 - https://artifacts.picoctf.net/c/15/level2.py
Resolving artifacts.picoctf.net (artifacts.picoctf.net)... 18.172.21.90, 18.172.21.43, 18.172.21.12, ...
Connecting to artifacts.picoctf.net (artifacts.picoctf.net)|18.172.21.90|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 914 [application/octet-stream]
Saving to: 'level2.py'
level2.py 100%[=====] 914 --KB/s in 0.009s
2020-01-27 05:53:54 (97.4 KB/s) - 'level2.py' saved [914/914]

--(kali@kali): /media/sf_Downloaded_Cybersecfiles
$ wget https://artifacts.picoctf.net/c/15/level2.flag.txt.enc
2020-01-27 05:54:12 - https://artifacts.picoctf.net/c/15/level2.flag.txt.enc
Resolving artifacts.picoctf.net (artifacts.picoctf.net)... 18.172.21.26, 18.172.21.12, 18.172.21.43, ...
Connecting to artifacts.picoctf.net (artifacts.picoctf.net)|18.172.21.26|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 31 [application/octet-stream]
Saving to: 'level2.flag.txt.enc'
level2.flag.txt.enc 100%[=====] 31 --KB/s in 0s
2020-01-27 05:54:13 (29.7 MB/s) - 'level2.flag.txt.enc' saved [31/31]
```

- Ensured that both files were placed in the same directory, as required by the script.

```
(kali@kali)-[/media/sf_Downloaded_Cybersecfiles]
$ ls
level2.flag.txt.enc level2.py
```

- Opened the Linux Terminal and navigated to the directory containing the files.
- Used the **nano** command to inspect the Python script and analyze its contents. (CTRL + X to exit)

```
(kali@kali)-[/media/sf_Downloaded_Cybersecfiles]
$ nano level2.py
```

- Observed an if condition that validates the entered password, noting that the expected value was stored in **hexadecimal format**.

```
GNU nano 8.4 level2.py
### THIS FUNCTION WILL NOT HELP YOU FIND THE FLAG --LT #####
def str_xor(secret, key):
    #extend key to secret length
    new_key = key
    i = 0
    while len(new_key) < len(secret):
        new_key = new_key + key[i]
        i = (i + 1) % len(key)
    return ''.join([chr(ord(secret_c) ^ ord(new_key_c)) for (secret_c, new_key_c) in zip(secret, new_key)])

flag_enc = open('level2.flag.txt.enc', 'rb').read()

def level_2_pw_check():
    user_pw = input("Please enter correct password for flag: ")
    if( user_pw == chr(0x33) + chr(0x39) + chr(0x63) + chr(0x65) ):
        print("Welcome back... your flag, user:")
        decryption = str_xor(flag_enc.decode(), user_pw)
        print(decryption)
        return
    print("That password is incorrect")

level_2_pw_check()
```



- Identified that the script decrypts and prints the contents of the encrypted text file only if the correct password is provided.

```
user_pw = input("Please enter correct password for flag: ")
if( user_pw == chr(0x33) + chr(0x39) + chr(0x63) + chr(0x65) ):
    print("Welcome back... your flag, user:")
    decryption = str_xor(flag_enc.decode(), user_pw)
    print(decryption)
    return
```

- Used an online hexadecimal-to-ASCII converter to decode the hex value found in the script.

ASCII text

39ce

Hex (bytes)

chr(0x33) + chr(0x39) + chr(0x63) + chr(0x65)

Binary (bytes)

00001100 00110011 00001100 00111001 00001100 01100011 00001100 01100101

Decimal (bytes)

12 51 12 57 12 99 12 101

- Executed the Python script using the python3 command.

```
$ python3 level2.py
Please enter correct password for flag:
```

- Entered the correct password when prompted.

```
(kali㉿kali)-[/media/sf_Downloaded_Cybersecfiles]
$ python3 level2.py
Please enter correct password for flag: 39ce
Welcome back... your flag, user:
```

- Retrieved and submitted the flag displayed by the script.

V. Result

picoCTF{tr45h_51ng1ng_502ec42e}



```
(kali@kali)-[/media/sf_Downloaded_Cybersecfiles]
$ python3 level2.py
Please enter correct password for flag: 39ce
Welcome back... your flag, user:
picoCTF{tr45h_51ng1ng_502ec42e}
```

VI. Explanation

This challenge demonstrates how **encoding can be used to obscure sensitive information** within source code. Although the password was not immediately readable, it was stored in hexadecimal format, which is a reversible encoding method rather than encryption.

By converting the hexadecimal value into its ASCII representation, the original password was revealed. Once the correct password was entered, the Python script successfully decrypted the encrypted text file and displayed the flag.

The requirement that both files be located in the same directory is due to the script referencing the encrypted file using a **relative file path**. If the files were separated, the script would fail to locate the encrypted file.

From a security standpoint, this challenge illustrates why **storing encoded passwords directly in source code is insecure**, as attackers can easily decode them by inspecting the program.

VII. Conclusion

The PW Crack 2 challenge reinforced essential skills in **source code analysis, hexadecimal decoding, and basic authentication logic**. It highlighted the difference between encoding and encryption and demonstrated how weak password protection mechanisms can be easily bypassed. Understanding these concepts is fundamental for both CTF competitions and real-world secure software development.

— The Analyst: Hyposelenia